

Homofidäl

An Amharic homophone checker for the Ge'ez script

ሀ · ሐ · ገ ሰ · ሠ ጸ · ፀ አ · ዐ

Product	Homofidäl — repo/package slug: homofidel
Platform	Flutter (Android + Web build for the trial)
Backend	None required for the prototype (fully offline / on-device)
Prepared by	Mikiyas T. (Melloss) — Full-stack Flutter & backend developer
Context	Upwork fixed-price prototype trial (\$20), scoped as a proof of concept
Document date	17 July 2026
Status	Draft for pre-build alignment — scope to be confirmed before work begins

This document is the working context for building the prototype: the problem, the linguistics, the detection algorithm, the Flutter architecture, and the exact deliverables. It is intentionally scoped tight — a tiny, honest proof of concept, not a production app.

1 Overview & problem

Amharic is written in the Ge'ez script (ፊደል / *fidäl*), an abugida where each consonant has seven vowel orders. A quirk of the script is that several **different** base letters are pronounced **identically** in modern Amharic. When you write, you constantly choose between letters that sound the same but are spelled differently — and it's easy to pick the wrong one.

Because the wrong choice is still a **real, valid letter**, the result is a correctly-formed word that is simply the wrong spelling. This is the Amharic equivalent of English *their* / *there* / *they're* — except the confusable set is baked into the alphabet itself and touches a large share of everyday words.

Who it's for

Anyone typing Amharic who wants their spelling to look right: writers, students, content creators, and — honestly — me. I make these slips myself when writing captions, documents, and messages, which is why I'd actually use this.

What exists today, and the gap

Amharic spell checkers do exist. The most visible is **Sewasew** (sewasew.com), a free online checker backed by a dictionary of roughly 150,000+ words; there are also academic correctors and an open-source `amharic_spell_corrector` built on the SymSpell / edit-distance approach. These are good at catching **non-words** (a string that isn't in the dictionary).

The gap: homophone slips are **real-word → real-word** substitutions. Swapping ሰ for ሠ usually produces another string a dictionary happily accepts, so pure spell checkers wave it through. Nobody offers a small, focused tool that targets *only* this one error class and makes the correct choice one tap away. That's the whole idea of Homofidäl.

2 Goals & non-goals

Scope discipline is the point of a \$20 trial. This keeps us honest about what "done" means.

In scope (the prototype proves this)	Explicitly out of scope (for now)
• Paste or type Amharic text • Detect characters that belong to a homophone family • Highlight those characters as "choice points" • Tap a highlighted letter → see its same-sound siblings → tap to swap it in • Works fully offline, on-device	• A full Amharic dictionary / grammar checker • Autocorrect or "fix everything" one-click • System-wide keyboard / IME integration • Accounts, cloud sync, analytics • iOS release, Play Store publishing, polish pass

One-line definition of done: a working Flutter app where a user pastes Amharic text, sees the homophone-risk letters highlighted, and can tap any of them to view and apply the same-sound alternatives — delivered with source, run instructions, screenshots, and an honest notes file.

3 How it works, in one paragraph

Every Ge'ez letter is a single Unicode code point in a highly regular block. Homophone families are just small sets of **base letters** that share a sound. Given any character, the app checks whether it sits inside one of those families; if it does, the character is a "choice point." Swapping to a same-sound sibling is pure arithmetic on the code point — keep the vowel order, change the base. No machine learning, no network, no dictionary needed for the core interaction. A smarter "which one is probably correct" layer is possible later by checking candidates against a word-frequency list, but the prototype deliberately ships the deterministic, offline core first.

4 The homophone families

Modern Amharic collapses several historically-distinct consonants into the same pronunciation. The prototype targets the four canonical families below. Each base letter is shown in all seven vowel orders (ä u i a e ə o).

Family (sound)	Base letters	Seven orders	Unicode bases
H — /h/	ሀ ሐ ኀ	ሀሀ'ሂ'ሄህሆ ሐሐ'ሒ'ሔሐሕ ኀኀ'ኂ'ኄኀኆ	U+1200 U+1210 U+1280
S — /s/	ሰ ሠ	ሰሰ'ሲ'ሴሰሶ ሠሠ'ሳ'ሷሠሢ	U+1230 U+1220
Ts' — /ts'/	ጸ ፀ	ጸጸ'ጺጺጺጺ ፀፀ'፱፱፱፱	U+1338 U+1340
A — /ʔ ~ a/	አ ዐ	አአ'ኢ'ኣአ ዐዐ'ዒ'ዔዐ	U+12A0 U+12D0

Everyday examples where the choice matters: ሰላም (peace/hello), ለበበ (a name), ጸሎት (prayer), ፀሐይ (sun), ዐይን (eye). The prototype flags the choice points; it does not claim to be the authority on which spelling is "correct" — that judgment is the optional smart layer described in §5.

5 Detection engine

The engine is a **pure Dart library** with no Flutter or UI dependencies, so it can be unit-tested in isolation and reused later (e.g. inside a keyboard). It exposes one main function: given a string, return the list of flagged characters with their position and same-sound alternatives.

The core insight: it's arithmetic

Within a family, every base letter's seven orders are seven consecutive code points. So for any character, the vowel *order* is simply its offset from its base ($order = codePoint - base$), and a same-sound sibling is $otherBase + order$. Verified against the Unicode tables — a few examples:

```
// keep the vowel order, swap the base letter
ሳ (U+1233, S-family, order 3 "a") → siblings: [ ሣ (U+1223) ]
ቃ (U+1283, H-family, order 3 "a") → siblings: [ ሃ (U+1203), ሐ (U+1213) ]
ጸ (U+133B, Ts'-family, order 3 "a") → siblings: [ ጥ (U+1343) ]
```

Algorithm (pseudocode)

```
const families = [
  [0x1200, 0x1210, 0x1280], // H:  ሀ ሐ ሳ
  [0x1230, 0x1220],        // S:  ሰ ሠ
  [0x1338, 0x1340],        // Ts': ጸ ፀ
  [0x12A0, 0x12D0],        // A:  አ ፈ
];

function scan(text):
  flags = []
  for (index, char) in runes(text):
    cp = codePoint(char)
    for family in families:
      for base in family:
        if base <= cp <= base + 6: // char is in this family
          order = cp - base
          siblings = [chr(b + order) for b in family if b != base]
          flags.add({ index, char, family, order, siblings })
  return flags
```

Two modes — and what the prototype ships

Mode	What it does	Data needed
A. Choice-point highlighting MVP — ships	Flags every character that <i>could</i> be a homophone slip and offers the same-sound swaps on tap. Deterministic, instant, offline. Honest framing: "here are your choice points; pick the right letter."	None
B. Likely-error correction stretch goal	For each word, generate its homophone variants and check them against a bundled word-frequency list. If the typed form is rare/absent but a variant is common, surface that variant as a <i>suggested correction</i> (a stronger highlight) instead of just a neutral choice point. Cuts noise dramatically.	Offline word-frequency list (bundled asset)

Recommendation: ship **Mode A** as the guaranteed deliverable — it fully satisfies the agreed interaction and carries zero data risk. Attempt **Mode B** only if time remains, using a small public Amharic frequency list bundled as a local asset. This keeps the \$20 trial safe and still leaves an obvious, valuable "next step" to show the idea has legs. **Open decision to confirm before build:** Mode A only, or Mode A + attempt B?

6 User experience & flow

One screen, one clear interaction. No onboarding, no menus.

- Input.** A multi-line text field. User pastes or types Amharic. A አግሮኛ sample-text button pre-fills an example so the client can test in one tap.
- Check.** User taps **Check** (አረግጥ). The engine scans the text.
- Highlighted result.** The text renders below with every homophone choice point in a soft highlight. In Mode B, likely-error letters get a stronger (amber) highlight to separate "probably wrong" from "just a choice point."
- Tap to swap.** Tapping a highlighted letter opens a bottom sheet showing the same-sound siblings, each rendered large. Tapping a sibling replaces that exact character in the text and re-scans.
- Copy out.** A **Copy** button returns the corrected text to the clipboard.

The one genuinely tricky Flutter bit

You can't easily make individual characters inside a live `TextField` both colored and tappable while editing. The clean pattern for a prototype: **edit mode** is a normal `TextField`; pressing **Check** renders a read-only `RichText` where each flagged grapheme is a `TextSpan` with a `TapGestureRecognizer` (or a `WidgetSpan` for larger tap targets). Editing a letter mutates the underlying string and re-renders. Because every `fidäl` is a single Unicode code point, we iterate by `runes` — no grapheme-cluster gymnastics required.

7 Technical architecture

Deliberately small and layered so the interesting logic is decoupled from the UI.

Layer	Responsibility
Engine (pure Dart)	Homophone family tables, the <code>scan()</code> function, sibling generation, and (Mode B) the word-frequency lookup. No Flutter imports. Fully unit-tested.
State	Holds the current text, the scan results, and the selected letter. A single <code>ChangeNotifier / Provider</code> (or <code>Riverpod</code>) — BLoC would be overkill at this size.
UI	The one screen: input field, Check/Copy buttons, highlighted <code>RichText</code> result, and the swap bottom sheet.

Do we need a backend / Supabase?

No backend for the prototype. The entire core runs on-device with zero network calls — better for a demo (works offline, nothing to spin up) and it keeps the scope tiny. Supabase would only enter the picture *later* if we wanted cloud-hosted or user-contributed word lists, usage analytics, or crowd-sourced correction feedback. None of that is needed to prove the idea, so the trial ships without it.

8 Tech stack & dependencies

Concern	Choice	Why
Framework	Flutter (stable)	Native-feeling Android app; same code builds a web demo
Language	Dart	—
State management	Provider / Riverpod	Lightweight; right-sized for one screen
Text rendering	<code>RichText</code> / <code>TextSpan</code>	Per-character color + tap targets
Clipboard	Flutter services (built-in)	Copy corrected text out
Homophone data	Reuse my <code>amharic_homophones</code> package	The family mappings already exist — see Appendix B
Word list (Mode B only)	Bundled JSON asset	Offline frequency lookup, no network

Dependencies stay minimal on purpose — the goal is a clean, reviewable prototype, not a dependency tree.

9 Project structure

```
homofidel/
├── lib/
│   ├── main.dart           // app entry
│   ├── engine/
│   │   ├── families.dart   // the 4 family tables (Unicode bases)
│   │   ├── homophone_engine.dart // scan(), siblings(), pure Dart
│   │   └── word_checker.dart // Mode B (optional): frequency lookup
│   ├── state/
│   │   └── checker_state.dart // ChangeNotifier: text + results
│   └── ui/
│       ├── home_screen.dart // input, buttons, result
│       ├── highlighted_text.dart // RichText w/ tappable spans
│       └── swap_sheet.dart   // bottom sheet of siblings
├── assets/
│   └── word_freq.json       // Mode B only
├── test/
│   └── homophone_engine_test.dart // deterministic unit tests
├── README.md                // run instructions + notes
└── pubspec.yaml
```

10 Testing approach

The engine is deterministic, which makes it pleasant to test. The notes file will state exactly how it was verified.

- **Unit tests** on the engine: known letter → expected siblings for all four families and several vowel orders; non-Ge'ez text (Latin, digits, punctuation) produces zero flags; empty input is safe.
- **Boundary checks:** characters just outside a family's 7-order range are *not* flagged (e.g. the 8th-order labiovelar variants), so we don't over-flag.
- **Manual UX pass** on the sample text: highlight → tap → swap → copy, on both the Android build and the web build.

11 Deliverables

Mapped directly to what the client asked for in the job post:

Client asked for	How it's delivered
Source code via Git or agreed archive	Public/shared Git repo, clean commit history
Working prototype of the central interaction	The highlight → tap → swap flow, working on Android + web
Linux / web run instructions	README with exact flutter commands (web run works on Linux with no device)
Screenshots or short demo	Screenshots of each state + a short screen recording
Concise note: works / incomplete / tested / next	A short NOTES.md: what works, what's unfinished, how it was tested, what I'd try next — no pretending unfinished parts work

12 Build & run

The client works in Linux and asked for web/Linux run instructions; Flutter web needs no device or emulator, so it runs anywhere Flutter is installed.

```
# run the web build (works on Linux, no device needed)
flutter pub get
flutter run -d chrome

# build a shareable Android APK
flutter build apk --release
# -> build/app/outputs/flutter-apk/app-release.apk

# run the engine's unit tests
flutter test
```

13 Milestones (rough)

Step	Work
1 — Engine	Family tables + scan() + siblings, with unit tests. The whole idea proven in pure Dart before any UI.
2 — Screen	Input field, Check/Copy, highlighted RichText result.
3 — Swap	Tappable letters + bottom sheet + replace-and-rescan.
4 — Polish + deliver	Sample text, Android + web builds, screenshots, README, NOTES.md.
5 — If time	Attempt Mode B (word-frequency likely-error layer).

14 Risks & open questions

- **Noise in Mode A.** Common letters like ሰ and ለ appear constantly, so flagging every choice point can look busy. Mitigation: subtle highlight for choice points, reserve the strong highlight for Mode B's likely errors; let the user toggle highlighting off.
- **"Correct" is contextual.** Without a dictionary, the app can't say which spelling is right — only offer the options. That's why Mode A is framed as an assist, not an authority. Stated plainly in NOTES.md.
- **Scope creep (Android added).** Building Android on top of the web prototype is more work for the same fixed trial; keeping the core interaction identical across both keeps it contained.
- **Open decision:** Mode A only, or Mode A + attempt Mode B? Confirm before build so the estimate is honest.

15 Beyond the trial

Natural next steps if the idea proves useful — and a good fit for the ongoing, experimental work stream:

- Mode B tuning with a larger, better frequency corpus for genuine correction quality.
- More confusable sets beyond the core four (dialect-dependent overlaps).
- A custom keyboard / IME so correction happens as you type, system-wide.
- Publishing the engine as its own open-source Dart package — a sibling to amharic_homophones, ethio_numbers, and zema_text.

A Appendix — the name

Homofidäl = *homophone* + ፊደል (*fidäl*, "letter/alphabet"). It says exactly what the app does — same-sound letters — and nods to the script it works on. ASCII-safe slug for the repo and any future package: homofidel.

Alternate	Note
Ge’ezGuard	Clear and Western-friendly; "guards" your Ge'ez spelling. Less tied to the exact mechanic.
Fidäl Check	Plainest option; instantly understandable, less distinctive.

B Appendix — reused assets

The linguistic groundwork isn't starting from zero. I already maintain open-source Dart/Flutter packages in this exact space, which is why "the data exists" — the prototype is mostly wiring known mappings to a clean interaction:

Package	Relevance
amharic_homophones	The homophone family mappings — reused as the engine's data source
ethio_numbers	Ethiopic number formatting — same Ge'ez-tooling family, shows domain depth
zema_text	Ge'ez / liturgical text handling — same ecosystem

All at github.com/melloss.